Ron Vincent Cada

Student Enrollment Database – Design, Build & Query

PART 1 – Schema Design & Creation

```sql
create table students (
    id serial primary key,
    first_name varchar(250) not null,
    last_name varchar(250) not null,
    year int check (year >= 1 and year <= 4) not null,
    gender varchar(50),
    enrolled boolean default true,
    created_at timestamptz default now()
);
```

Statistics 1

Name	Value
Updated Rows	0
Execute time	0.024s
Start time	Mon May 11 14:57:02 PST 2026
Finish time	Mon May 11 14:57:02 PST 2026
Query	create table students (id serial primary key, first_name varchar(250) not null, last_name varchar(250) not null, year int check (year >= 1 and year <= 4) not null, gender varchar(50), enrolled boolean default true, created_at timestamptz default now())

```sql
create table programs (
    id serial primary key,
    program_name varchar(150) not null unique
)
```

Statistics 1

Name	Value
Updated Rows	0
Execute time	0.005s
Start time	Mon May 11 14:57:22 PST 2026
Finish time	Mon May 11 14:57:22 PST 2026
Query	create table programs (id serial primary key, program_name varchar(150) not null unique)

```sql
create table sections (
    id serial primary key,
    code varchar(50) not null unique,
    year int not null,
    program_id int not null references programs(i
)
```

Statistics 1

Name	Value
Updated Rows	0
Execute time	0.008s
Start time	Mon May 11 14:57:56 PST 2026
Finish time	Mon May 11 14:57:56 PST 2026
Query	create table sections (id serial primary key, code varchar(50) not null unique, year int not null, program_id int not null references programs(id))

```sql
create table student_sections (
    id serial primary key,
    student_id int not null references students(id),
    section_id int not null references sections(id),
    enrolled_at timestamptz default now()
)
```

Statistics 1

Name	Value
Updated Rows	0
Execute time	0.003s
Start time	Mon May 11 14:59:13 PST 2026
Finish time	Mon May 11 14:59:13 PST 2026
Query	create table student_sections (id serial primary key, student_id int not null references students(id), section_id int not null references sections(id), enrolled_at timestamptz default now())

```sql
create table student_grades (
    id serial primary key,
    student_id int not null references students(id),
    grade int check (grade >= 0 and grade <= 100)
)
```

Statistics 1

Name	Value
Updated Rows	0
Execute time	0.016s
Start time	Mon May 11 14:59:38 PST 2026
Finish time	Mon May 11 14:59:38 PST 2026
Query	create table student_grades (
	id serial primary key,
	student_id int not null references students(id),
	grade int check (grade >= 0 and grade <= 100)
	)

PART 2 – Seed Data & INSERT

Insert data in students table

```sql
insert into students (first_name, last_name, year, gender, enrolled)
values
('Ron Vincent', 'Cada', 3, 'male', true),
('Mika Andrea', 'Gomez', 2, 'female', true),
('Roosc', 'Zano', 1, 'male', false),
('Kurt', 'Laja', 3, 'male', true)
returning id
```

students 1

insert into students (first_name, last_name, year, gender, e | Enter a SQL expression to filter resul

Grid | Text

	123 id
1	1
2	2
3	3
4	4

Verify the count of the students table

```sql
select count(*) from students
```

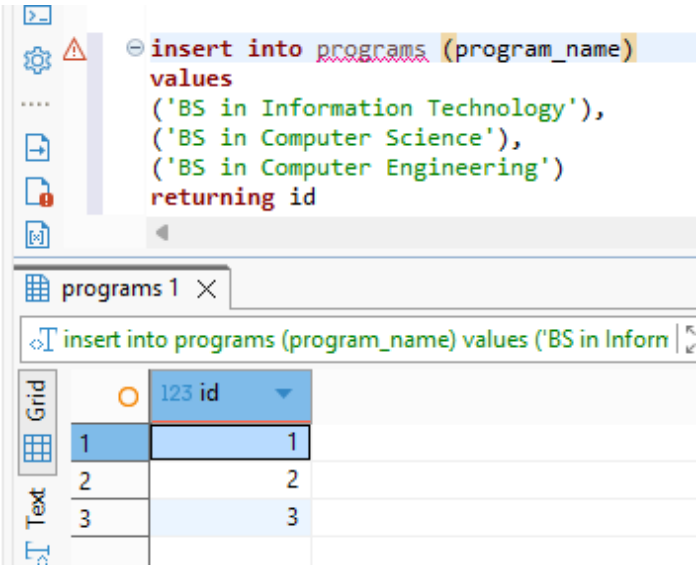
Results 1

select count(*) from students | Enter a SQL expres

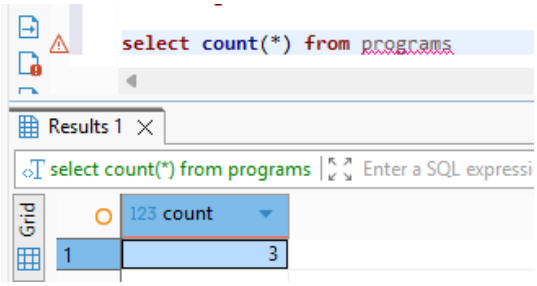
Grid

	123 count
1	4

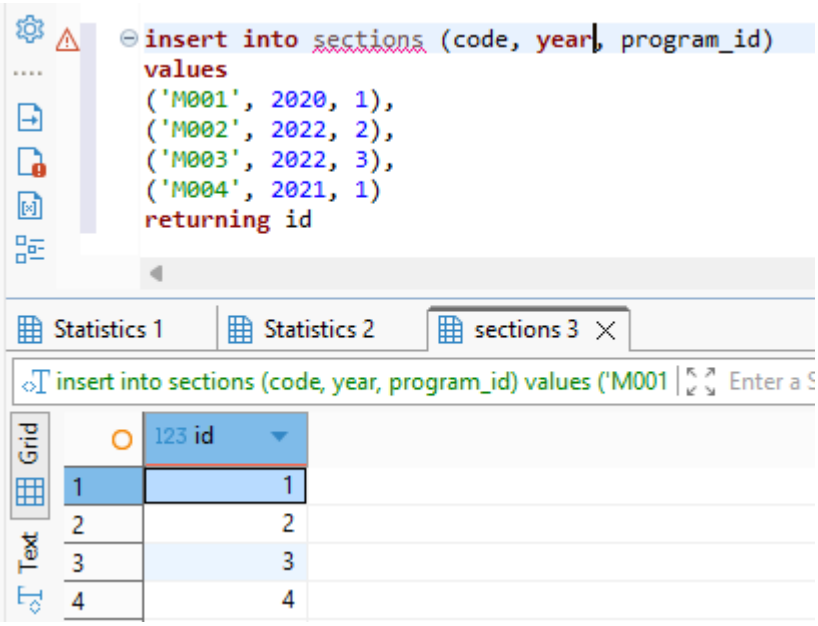
Insert data in programs table



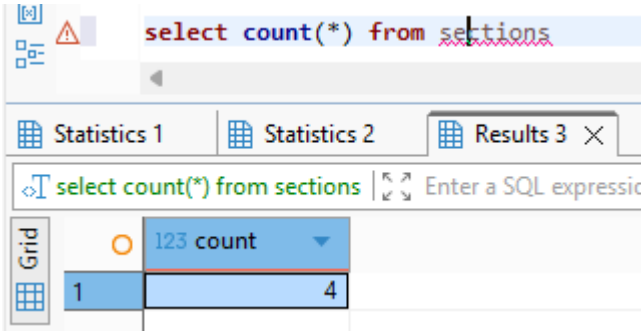
Verify the count of the programs table



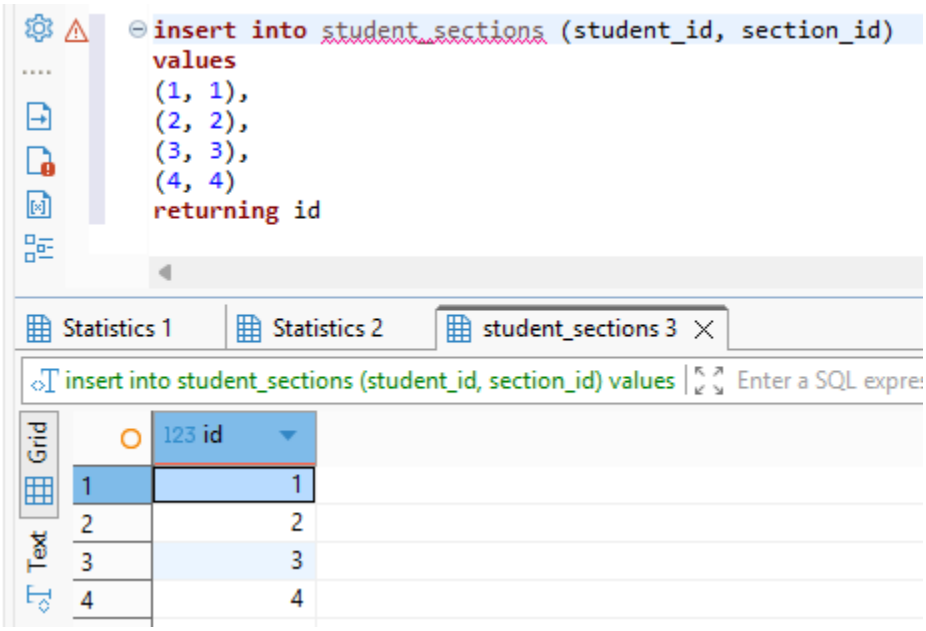
insert data in sections table



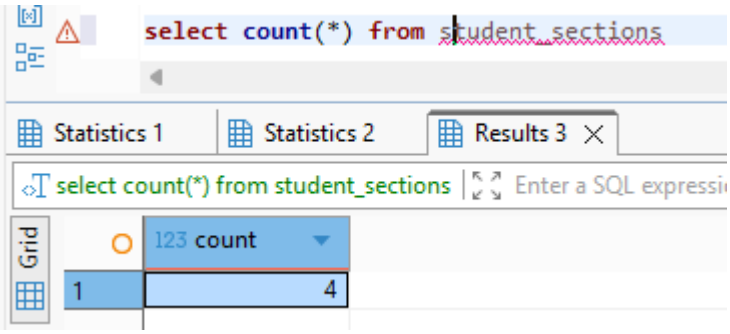
verify the count of sections table



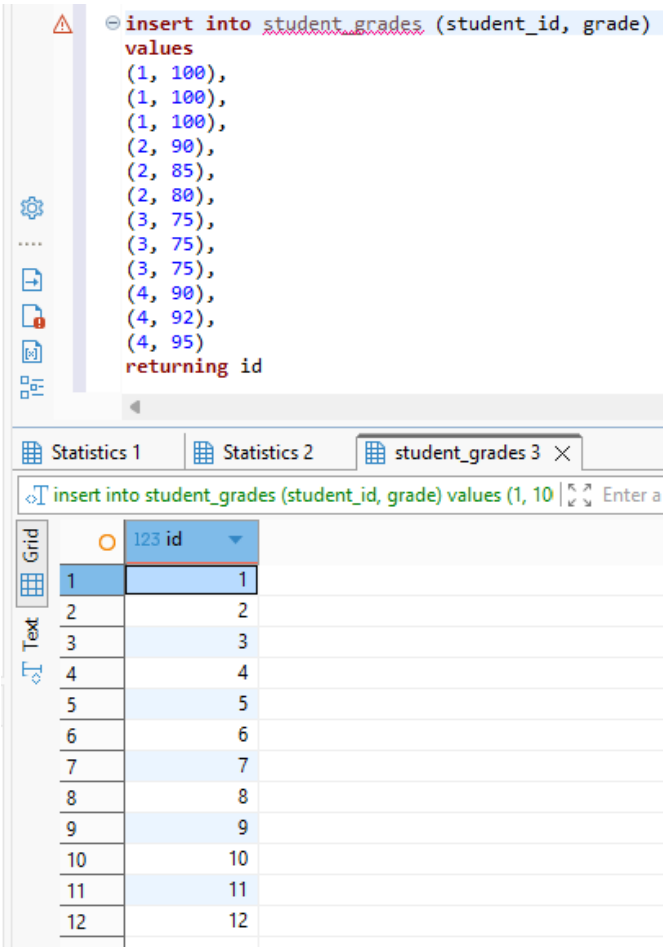
Insert data in student_sections junction table



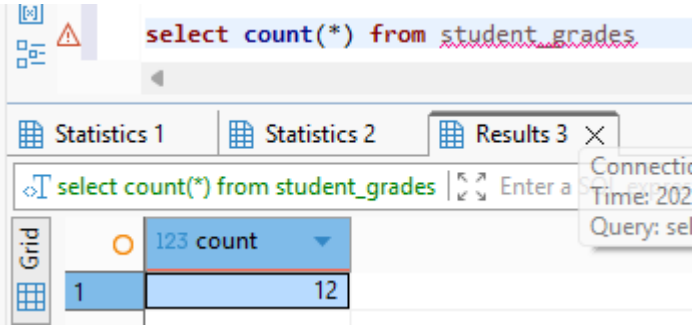
Verify the count of student_sections junction table



Insert data in student_grades



Verify the count of student_grades



Part 3 – Queries

display all students with their program names

```
select s.id, s.first_name, s.last_name, s.year, s.gender, p.program_name
from students s
join student_sections ss on s.id = ss.student_id
join sections sec on ss.section_id = sec.id
join programs p on sec.program_id = p.id
```

Statistics 1 | Statistics 2 | students(+) 3

select s.id, s.first_name, s.last_name, s.year, s.gender, p.prc — Enter a SQL expression to filter results (use Ctrl+Space)

	id	first_name	last_name	year	gender	program_name
1	1	Ron Vincent	Cada	3	male	BS in Information Technology
2	2	Mika Andrea	Gomez	2	female	BS in Computer Science
3	3	Roosc	Zano	1	male	BS in Computer Engineering
4	4	Kurt	Laja	3	male	BS in Information Technology

display students with their assigned section codes

```
select s.id, s.first_name, s.last_name, s.year, s.gender, p.program_name, sec.code
from students s
join student_sections ss on s.id = ss.student_id
join sections sec on ss.section_id = sec.id
join programs p on sec.program_id = p.id
```

Statistics 1 | Statistics 2 | students(+) 3

select s.id, s.first_name, s.last_name, s.year, s.gender, p.prc — Enter a SQL expression to filter results (use Ctrl+Space)

	id	first_name	last_name	year	gender	program_name	code
1	1	Ron Vincent	Cada	3	male	BS in Information Technology	M001
2	2	Mika Andrea	Gomez	2	female	BS in Computer Science	M002
3	3	Roosc	Zano	1	male	BS in Computer Engineering	M003
4	4	Kurt	Laja	3	male	BS in Information Technology	M004

Retrieve all the currently enrolled students

In here student Roosc Zano is not shown since he is not enrolled

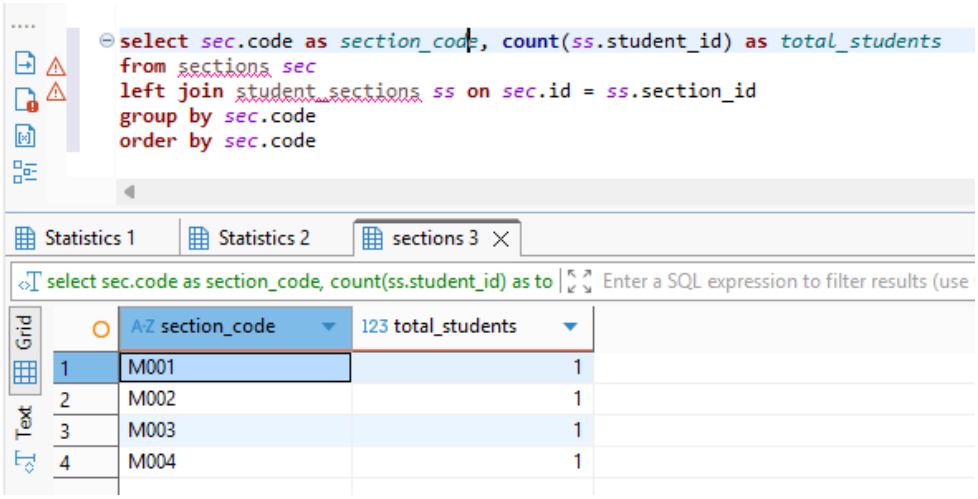
```
select * from students
where enrolled = true
```

Statistics 1 | Statistics 2 | students 3

select * from students where enrolled = true — Enter a SQL expression to filter results (use Ctrl+Space)

	id	first_name	last_name	year	gender	enrolled	created_at
1	1	Ron Vincent	Cada	3	male	[v]	2026-05-11 15:08:43.068 +0800
2	2	Mika Andrea	Gomez	2	female	[v]	2026-05-11 15:08:43.068 +0800
3	4	Kurt	Laja	3	male	[v]	2026-05-11 15:08:43.068 +0800

Count the number of students per section using GROUP BY



```
select sec.code as section_code, count(ss.student_id) as total_students
from sections sec
left join student_sections ss on sec.id = ss.section_id
group by sec.code
order by sec.code
```

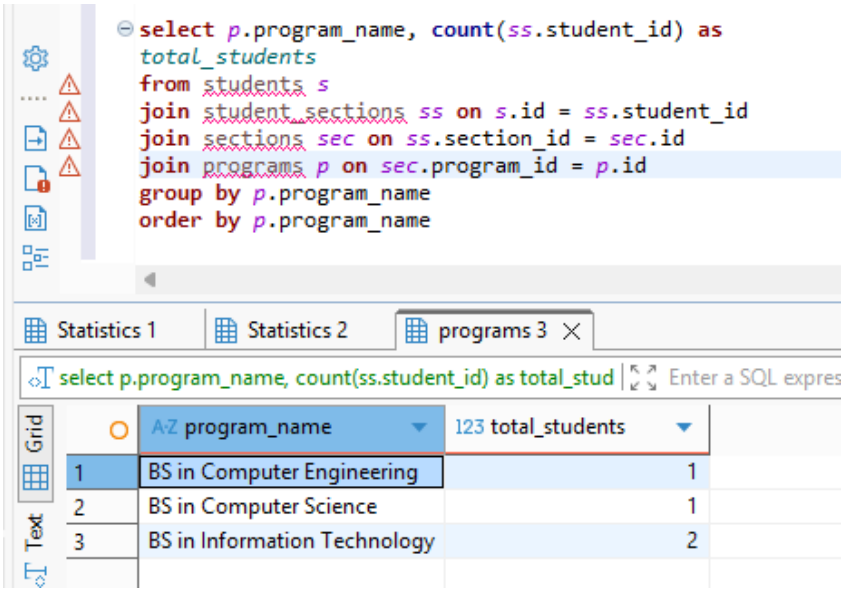
Statistics 1 | Statistics 2 | sections 3

select sec.code as section_code, count(ss.student_id) as to | Enter a SQL expression to filter results (use

Grid | Text

	section_code	total_students
1	M001	1
2	M002	1
3	M003	1
4	M004	1

Count the number of students per program



```
select p.program_name, count(ss.student_id) as
total_students
from students s
join student_sections ss on s.id = ss.student_id
join sections sec on ss.section_id = sec.id
join programs p on sec.program_id = p.id
group by p.program_name
order by p.program_name
```

Statistics 1 | Statistics 2 | programs 3

select p.program_name, count(ss.student_id) as total_stud | Enter a SQL expres

Grid | Text

	program_name	total_students
1	BS in Computer Engineering	1
2	BS in Computer Science	1
3	BS in Information Technology	2

Computing the average grade of each student using AVG

```
select s.id, s.first_name, s.last_name, round(avg(g.grade), 2) as average_grade
from students s
left join student_grades g on s.id = g.student_id
group by s.id, s.first_name, s.last_name
order by average_grade desc
```

Statistics 1 | Statistics 2 | students 3

select s.id, s.first_name, s.last_name, round(avg(g.grade), 2

Enter a SQL expression to filter results (use Ctrl+Spa

Grid | Text

	123 id	A-Z first_name	A-Z last_name	123 average_grade
1	1	Ron Vincent	Cada	100
2	4	Kurt	Laja	92.33
3	2	Mika Andrea	Gomez	85
4	3	Roosc	Zano	75

Finding students who are not assigned to any section using LEFT JOIN with IS NULL

In here there are no students found with no sections assigned

```
select s.id, s.first_name, s.last_name
from students s
left join student_sections ss on s.id = ss.student_id
where ss.student_id is null
```

Statistics 1 | Statistics 2 | students 3

select s.id, s.first_name, s.last_name from students s left jo

Enter a SQL expressior

Grid | Text

	123 id	A-Z first_name	A-Z last_name

Finding sections with no students assigned

In here there are no sections found with no students assigned

```
select sec.id, sec.code, sec.year
from sections sec
left join student_sections ss on sec.id = ss.section_id
where ss.section_id is null
```

Statistics 1 | Statistics 2 | sections 3

select sec.id, sec.code, sec.year from sections sec left join

Enter a SQL expression t

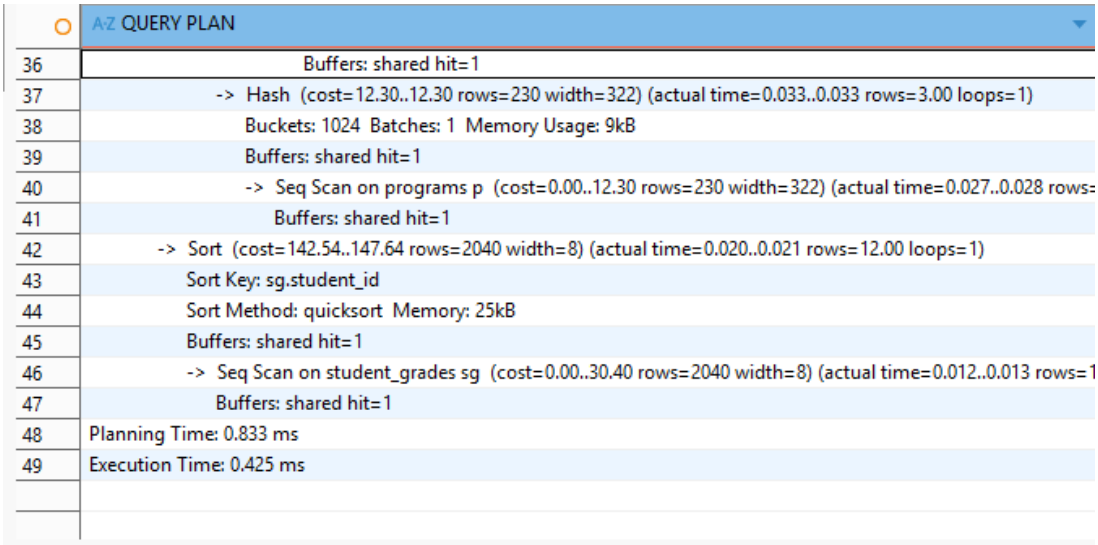
Grid | Text

	123 id	A-Z code	123 year

PART 4 – Indexes & Constraints

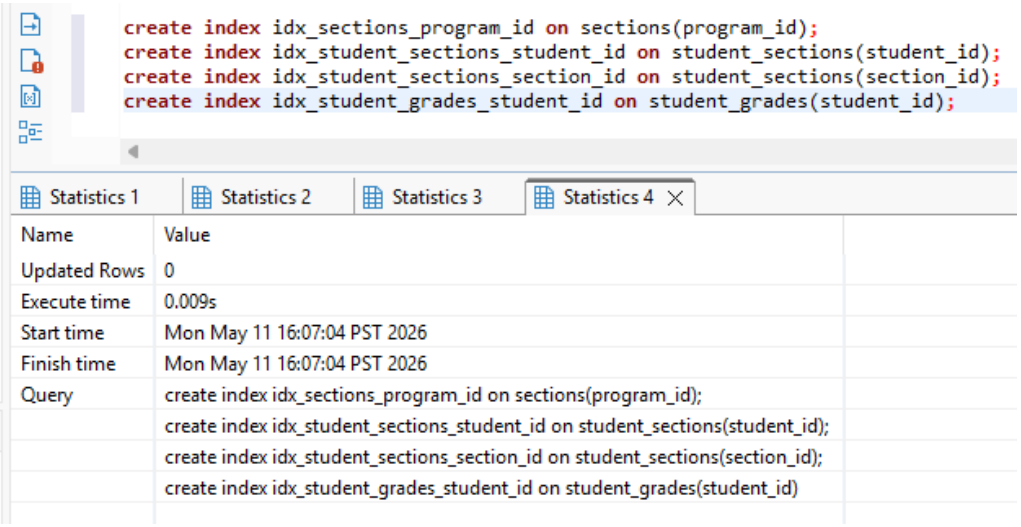
```
explain analyze
select s.first_name, s.last_name, p.program_name, sec.code as section_code, round(avg(sg.grade), 2) as average_grade
from students s
join student_sections ss on s.id = ss.student_id
join sections sec on ss.section_id = sec.id
join programs p on sec.program_id = p.id
left join student_grades sg on s.id = sg.student_id
group by s.id, s.first_name, s.last_name, p.program_name, sec.code
order by s.id
```

Before creating Indexes



	QUERY PLAN
36	Buffers: shared hit=1
37	-> Hash (cost=12.30..12.30 rows=230 width=322) (actual time=0.033..0.033 rows=3.00 loops=1)
38	Buckets: 1024 Batches: 1 Memory Usage: 9kB
39	Buffers: shared hit=1
40	-> Seq Scan on programs p (cost=0.00..12.30 rows=230 width=322) (actual time=0.027..0.028 rows=
41	Buffers: shared hit=1
42	-> Sort (cost=142.54..147.64 rows=2040 width=8) (actual time=0.020..0.021 rows=12.00 loops=1)
43	Sort Key: sg.student_id
44	Sort Method: quicksort Memory: 25kB
45	Buffers: shared hit=1
46	-> Seq Scan on student_grades sg (cost=0.00..30.40 rows=2040 width=8) (actual time=0.012..0.013 rows=1
47	Buffers: shared hit=1
48	Planning Time: 0.833 ms
49	Execution Time: 0.425 ms

create indexes



```
create index idx_sections_program_id on sections(program_id);
create index idx_student_sections_student_id on student_sections(student_id);
create index idx_student_sections_section_id on student_sections(section_id);
create index idx_student_grades_student_id on student_grades(student_id);
```

Statistics 1 | Statistics 2 | Statistics 3 | Statistics 4

Name	Value
Updated Rows	0
Execute time	0.009s
Start time	Mon May 11 16:07:04 PST 2026
Finish time	Mon May 11 16:07:04 PST 2026
Query	create index idx_sections_program_id on sections(program_id);
	create index idx_student_sections_student_id on student_sections(student_id);
	create index idx_student_sections_section_id on student_sections(section_id);
	create index idx_student_grades_student_id on student_grades(student_id)

After creating indexes